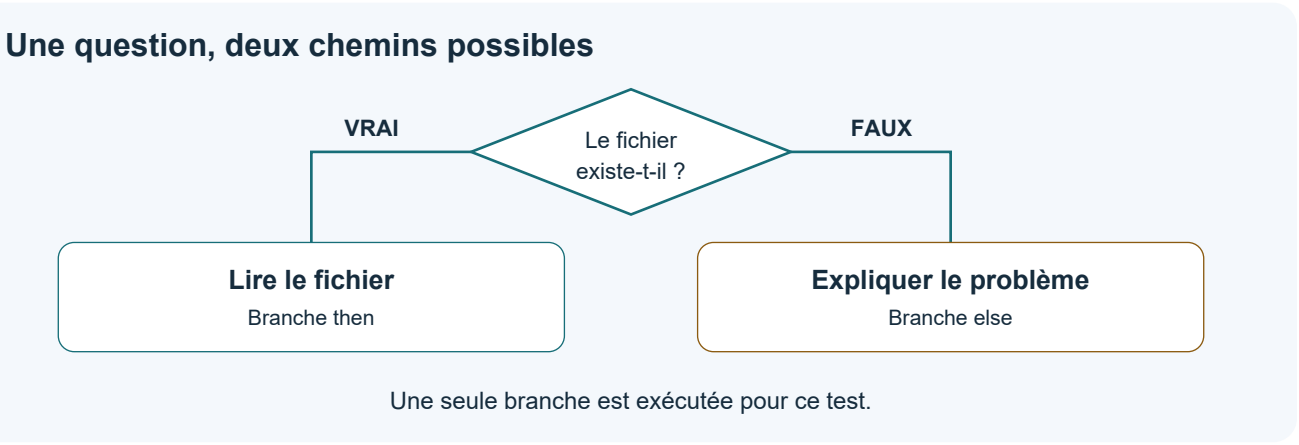
Chapitre 04 — Les tests et les conditions

Objectif Faire prendre une décision à ton script : continuer, choisir une action ou expliquer pourquoi il s'arrête.

1. Une question avec deux chemins



Un test choisit entre deux actions

`if` lance une commande, lit son **code de retour** et choisit un chemin en conséquence.

```
destination="Orion"
if [[ "$destination" == "Orion" ]]; then
    echo 'Bonne destination'
else
    echo 'Verifier le billet'
fi
```

Résultat : `Bonne destination`. Avec `destination="Vega"`, tu prendrais l'autre chemin.

`then` ouvre le premier bloc, `else` introduit le second, `fi` ferme le `if`. L'indentation de quatre espaces rend les branches visibles.

Ici, `0` veut dire vrai Souviens-toi du chapitre 03 : une commande qui réussit renvoie le code `0`. `if` ouvre le bloc `then` quand la commande **réussit**, donc quand elle renvoie `0`. C'est l'inverse de ce que tu as peut-être vu ailleurs, où `0` signifie « faux ».

`[[ ... ]]` est une commande comme une autre : elle n'affiche rien, elle renvoie seulement un code.

```
[[ "Orion" == "Orion" ]]; echo "$?"   # 0 : le test réussit
[[ "Orion" == "Vega" ]];  echo "$?"   # 1 : le test échoue
```

C'est cette mécanique qui permettra, au §5, d'utiliser n'importe quelle commande comme test.

Dans ce cours, nous utilisons `[[ ... ]]`, une syntaxe de **Bash**. Garde les espaces après `[[` et avant `]]`. Tu croiseras souvent `[ ... ]`, avec un seul crochet : c'est la forme historique, commune à tous les shells. Elle fait le même travail avec davantage de pièges — en Bash, garde `[[ ... ]]`.

2. Comparer du texte

Reprenons la même variable pour poser les autres questions possibles :

Test	Question posée
`[[ "$destination" == "Orion" ]]`	La valeur est-elle exactement `Orion` ?
`[[ "$destination" != "Orion" ]]`	Est-elle différente d'`Orion` ?
`[[ -z "$destination" ]]`	La valeur est-elle vide ?
`[[ -n "$destination" ]]`	Contient-elle au moins un caractère ?

`-z` sert surtout juste après un `read` : il repère une réponse laissée vide.

```
read -p 'Destination : ' destination
if [[ -z "$destination" ]]; then
    echo 'Aucune destination saisie'
else
    echo "Cap sur $destination"
fi
```

Essaie les deux cas : tape un nom, puis relance en appuyant directement sur **Entrée**.

Le signe `=` d'une affectation n'est pas un test. `destination="Orion"` change la variable ; le test, lui, vérifie sa valeur sans la modifier.

3. Comparer des entiers

Pourquoi un second jeu d'opérateurs ? Parce qu'une variable Bash contient toujours du **texte**. Il n'existe pas de variable « nombre » d'un côté et « texte » de l'autre : c'est l'**opérateur** qui décide comment cette valeur est lue.

Opérateur	Signification
`-eq`, `-ne`	Égal, différent
`-lt`, `-le`	Inférieur, inférieur ou égal
`-gt`, `-ge`	Supérieur, supérieur ou égal

```
energie=30
if [[ "$energie" -ge 20 ]]; then
    echo 'Depart autorise'
else
    echo 'Recharge necessaire'
fi
```

Résultat : `Depart autorise`. Avec `energie=10`, tu prendrais l'autre chemin.

Le bon opérateur pour la bonne lecture Comparer des nombres avec les opérateurs de texte ne déclenche aucune erreur : seul le résultat change.

```
a=10
b=9
[[ "$a" -gt "$b" ]]   # VRAI : lus comme des nombres, 10 est plus grand que 9
[[ "$a" > "$b" ]]     # FAUX : lus comme du texte, « 1 » vient avant « 9 »
```

Retiens la règle : `==` et `!=` pour du texte, `-eq` et `-gt` pour des nombres.

On peut distinguer plusieurs situations avec `elif`. Voici la forme générale, à lire comme un modèle :

```
if premier_test; then
    # commandes si le premier test reussit
elif second_test; then
    # commandes si le premier echoue et que celui-ci reussit
else
    # commandes si aucun test ne reussit
fi
```

Tu peux enchaîner autant de `elif` que nécessaire. Le bloc `else` est facultatif : sans lui, un cas non prévu ne déclenche simplement rien.

Appliqué à l'énergie du vaisseau, en demandant la valeur plutôt qu'en la fixant dans le code :

```
read -p 'Energie disponible : ' energie

if [[ "$energie" -ge 50 ]]; then
    echo 'Grande mission'
elif [[ "$energie" -ge 20 ]]; then
    echo 'Petite mission'
else
    echo 'Recharge'
fi
```

Relance-le trois fois pour parcourir les trois branches :

```
Energie disponible : 60     → Grande mission
Energie disponible : 30     → Petite mission
Energie disponible : 10     → Recharge
```

Bash examine les branches de haut en bas et s'arrête à la première qui réussit : avec `60`, il ne teste même pas `-ge 20`.

Une saisie n'est pas forcément un nombre Réponds `banane` à la question, puis observe :

```
Energie disponible : banane
Recharge
```

Aucune erreur, aucun avertissement. Dans `[[ ... ]]`, un mot qui n'est pas un nombre est lu comme la valeur `0` — donc inférieur à 20. Le script répond quelque chose de faux sans prévenir.

C'est la limite d'un test numérique sur une saisie libre : il faut vérifier la valeur **avant** de la comparer. Le §6 installe ce réflexe — vérifier d'abord, agir ensuite — et le §7 fournit un outil pour le faire, puisqu'un

motif de `case` peut décrire la forme attendue.

4. Tester un fichier

```
mkdir -p ~/atelier-bash/conditions
cd ~/atelier-bash/conditions
echo 'ORION-OK' > billet.txt

if [[ -f "billet.txt" ]]; then
    echo 'Billet trouve'
else
    echo 'Billet absent'
fi
```

Test	Vérification
`-e`	Le chemin existe
`-f`	C'est un fichier ordinaire
`-d`	C'est un dossier
`-r`	Le fichier est lisible pour le processus
`-w`	Le fichier est modifiable pour le processus
`-s`	Le fichier existe et sa taille dépasse zéro

Un dossier n'est pas un fichier ordinaire. Choisis le test adapté plutôt que de tout vérifier avec `-e` : avant de lire un fichier, `-f` puis `-r` ; avant d'y écrire, `-w`.

5. Réagir au résultat d'une commande

Le §1 l'annonçait : `if` ne sait lire qu'un code de retour. On peut donc placer **n'importe quelle commande** après `if`, sans `[[ ... ]]`.

```
if grep -qx 'ORION-OK' billet.txt; then
    echo 'Billet accepte'
else
    echo 'Billet refuse'
fi
```

`grep -q` n'affiche pas les lignes trouvées. `-x` exige une correspondance sur une ligne entière. Le code `0` de `grep` ouvre le premier bloc.

Cet exemple vérifie qu'au moins une ligne est exactement `ORION-OK`. Ce n'est pas une vérification de signature ni un vrai système d'accès.

Même principe pour savoir si un outil est présent sur la machine :

```
if command -v curl > /dev/null; then
    echo 'curl est installe'
else
    echo 'curl est absent'
```

```
fi
```

`command -v` cherche la commande et renvoie `0` si elle existe. `> /dev/null` jette son affichage : seul le code de retour nous intéresse.

6. Vérifier les arguments avant de les utiliser

Contenu de `controle.sh` :

fichier: controle.sh

```
#!/usr/bin/env bash

if [[ "$#" -ne 1 ]]; then
    echo "Usage : $0 fichier" >&2
    exit 2
fi

fichier="$1"
if [[ ! -f "$fichier" ]]; then
    echo "Fichier absent : $fichier" >&2
    exit 2
fi

echo "Fichier disponible : $fichier"
exit 0
```

- `$#` vérifie le nombre d'arguments **avant** la lecture de `$1`.
- `!` inverse un test.
- `>&2` dirige un message vers la sortie d'erreur.
- `exit` termine ce script avec le code choisi.

Vérifie la syntaxe, puis lance les trois cas. Comme au chapitre 03, `$?` se lit **juste après** la commande concernée :

```
bash -n controle.sh

bash controle.sh billet.txt
echo "Code : $?"

bash controle.sh introuvable.txt
echo "Code : $?"

bash controle.sh
echo "Code : $?"
```

Tu obtiens dans l'ordre :

```
Fichier disponible : billet.txt
Code : 0
Fichier absent : introuvable.txt
Code : 2
Usage : controle.sh fichier
Code : 2
```

Le premier cas réussit. Le deuxième est arrêté par la seconde garde, le troisième par la première. Le code `2` est une convention choisie pour ce programme ; d'autres commandes en utilisent d'autres.

7. Choisir parmi plusieurs mots avec case

`case` compare une même valeur à plusieurs motifs successifs. Voici la forme générale, à lire comme un modèle :

```
case "$variable" in
    motif)
        # commandes si la valeur correspond a ce motif
        ;;
    autre_motif)
        # commandes si elle correspond a cet autre motif
        ;;
    *)
        # commandes pour tous les cas restants
        ;;
esac
```

Bash examine les motifs de haut en bas et s'arrête au premier qui correspond.

Appliqué à un choix de poste :

```
choix="radio"
case "$choix" in
    radio)
        echo 'Canal radio ouvert'
        ;;
    reserve)
        echo 'Reserve disponible'
        ;;
    *)
        echo 'Choix inconnu'
        ;;
esac
```

Chaque choix se termine par `)`. `;;` termine sa branche. `*` récupère les autres cas. `esac` ferme la structure.

Une même branche peut accepter plusieurs mots, séparés par `|` :

```
case "$choix" in
    radio | phonie)
        echo 'Canal radio ouvert'
        ;;
    reserve | stock)
        echo 'Reserve disponible'
        ;;
    *)
        echo 'Choix inconnu'
        ;;
esac
```

C'est l'intérêt principal de `case` face à une série de `elif` : regrouper les synonymes sans répéter la condition.

Un motif n'est pas forcément un mot entier : il peut décrire une **forme**. `[0-9]` n'accepte qu'un seul chiffre, `*.txt` qu'un nom finissant par `.txt`. C'est ce qui permet de valider une saisie avant de s'en servir, comme le demandait l'avertissement du §3 :

```
read -p 'Un chiffre de 0 a 9 : ' choix
case "$choix" in
    [0-9])
        echo "Chiffre accepte : $choix"
        ;;
    *)
        echo "Ce n'est pas un chiffre" >&2
        ;;
esac
```

8. Combiner deux vérifications

Dans un test

`&&` signifie « et » : les deux tests doivent réussir.

```
if [[ -f "billet.txt" && -r "billet.txt" ]]; then
    echo 'Le billet est un fichier lisible'
fi
```

`||` signifie « ou » : il suffit que l'un des deux réussisse.

```
destination="Vega"
if [[ "$destination" == "Orion" || "$destination" == "Vega" ]]; then
    echo 'Destination desservie'
else
    echo 'Destination non desservie'
fi
```

Résultat : `Destination desservie`. Avec `destination="Sirius"`, tu obtiendrais l'autre message.

Commence par des tests courts ; si la condition devient difficile à lire, sépare les vérifications et explique chaque erreur séparément.

Entre deux commandes

Les mêmes signes s'emploient aussi **entre deux commandes**, en dehors de tout `[[ ... ]]`. Le sens est identique, appliqué cette fois aux codes de retour du §1 :

- `A && B` : `B` ne s'exécute que si `A` a **réussi**.
- `A || B` : `B` ne s'exécute que si `A` a **échoué**.

```
mkdir -p rapports && echo 'Dossier pret'
grep -qx 'AUTRE-STATION' billet.txt || echo 'Billet non conforme' >&2
```

La première ligne n'affiche son message que si le dossier a bien été créé. La seconde signale un problème uniquement quand `grep` ne trouve rien.

C'est une écriture courte pour un `if` à une seule branche :

```
# ces deux ecritures font la meme chose
```

```
if ! grep -qx 'AUTRE-STATION' billet.txt; then
    echo 'Billet non conforme' >&2
fi

grep -qx 'AUTRE-STATION' billet.txt || echo 'Billet non conforme' >&2
```

Réserve-la aux cas courts : dès qu'il y a plusieurs commandes à enchaîner, le `if` reste plus lisible.

À retenir

- `if` lit le **code de retour** d'une commande : `0` ouvre le bloc `then`. C'est l'inverse de la convention d'autres langages.
- Une valeur est toujours du texte : `==` et `!=` la comparent comme du texte, `-eq` et `-gt` comme un nombre.
- Texte : `-z` pour une valeur vide, `-n` pour une valeur remplie.
- Fichiers : `-f` et `-d` pour le type, `-r` et `-w` pour les droits, `-s` pour un contenu non vide.
- N'importe quelle commande peut servir de test : `grep -q`, `command -v`, ou un script à toi.
- Vérifie les arguments et les fichiers avant leur utilisation ; prévois un message sur `>&2` et un code de retour avec `exit`.
- `case` regroupe plusieurs mots par branche avec `|`.
- `&&` exige les deux, `||` se contente d'un seul — dans un test comme entre deux commandes, où `A || B` ne lance `B` que si `A` a échoué.

À toi : quel test différencie un dossier d'un fichier ? Quelle branche exécute `case` pour un mot inconnu ? Pourquoi `if grep -q ...` fonctionne-t-il sans `[[ ... ]]` ?

Passer au TP